CS3423: Compilers I

Lab Evaluation – Lexical Analyzer

Department of Computer Science & Engineering
IIT Hyderabad

April 2026

Overview

This examination tests your understanding of lexical analysis. You will implement a scanner using `lex/flex` for a small language. The evaluation has four parts, each adding new token types. The exam is 2 hours. You must work alone. No AI tools, no collaboration, no pre-written code.

Evaluation

Total marks: 20 (12 objective + 8 subjective).

Objective (12 marks): 12 test cases (3 per part). One test case per part is public (given below). The other two are private. Each test case is worth 1 mark – exact match required.

Subjective (8 marks):

- Regular definitions and pattern organisation (2)
- Edge case handling (2)
- Code clarity and comments (2)
- `README.txt` (2) – briefly explain your approach, assumptions, and any known issues.

The four parts are cumulative. Part II requires everything from Part I, Part III requires Parts I+II, and Part IV requires all previous parts. A correct solution for Part IV will automatically pass the test cases of Parts I–III because it only adds new rules – it never changes or removes existing ones.

1 Language Specification

1.1 Labels

Every statement starts with a label of the form `pp<number>:` (e.g., `pp1:`, `pp42:`). The colon is part of the label. The prefix `pp` is reserved for labels; it cannot appear in identifiers. Token type: `LABEL`. Lexeme includes the colon.

1.2 Scopes and separators

Blocks use square brackets: `[` and `]`. Token types: `SCOPE_OPEN`, `SCOPE_CLOSE`. Statements end with semicolon: `SEPARATOR: ;`.

1.3 Comments

Single-line comments start with `//` and go to end of line. They produce no output.

1.4 Data types

Three keywords: `integer`, `character`, `string`. Token type: `DATATYPE`.

1.5 Identifiers

Two kinds:

- Standard: a letter followed by letters, digits, or underscores. Example: `x`, `myVar`, `count1`.
- Special-start: begins with one of `@#*-+/_:\`, then only letters, digits, underscores. Example: `/result`, `@id`, `_temp`.

Token type: `IDENTIFIER`.

1.6 Keywords

Single-word: `do`, `null`, `repeat`. Multi-word (exactly single spaces): `in case that`, `otherwise`, `jump to`. All produce `KEYWORD`.

1.7 Operators

Binary arithmetic: `+`, `-`, `*`, `/`, `%%` (modulo). Assignment: `=`. All these produce `OPERATOR`. Unary operator: `_` (square root). Token type: `UNARY_OP`.

Relational (keywords): `gt`, `lt`, `gteq`, `lteq` → `RELOP`. Logical (keywords): `and`, `or` → `LOGICAL`.

1.8 String literals

Double quotes. Escape sequences: `\n`, `\t`, `\\`, `\"`. Token type: `STRING`. Lexeme includes the quotes.

1.9 Numbers

One or more digits. Token type: `NUMBER`.

1.10 Whitespace

Spaces, tabs, newlines are ignored.

1.11 Errors

Any character that does not match a valid token produces `ERROR: X` where X is the character.

2 Output Format

Each token on its own line: `TOKEN_TYPE: lexeme`. Token types: `KEYWORD`, `IDENTIFIER`, `NUMBER`, `STRING`, `OPERATOR`, `UNARY_OP`, `RELOP`, `LOGICAL`, `LABEL`, `DATATYPE`, `SCOPE_OPEN`, `SCOPE_CLOSE`, `SEPARATOR`, `ERROR`.

Example:

```
LABEL: pp1:
DATATYPE: integer
IDENTIFIER: x
OPERATOR: =
NUMBER: 10
SEPARATOR: ;
```

3 Part I: Basic Tokens (3 marks)

In Part I you must recognise the following tokens. Your lexer should correctly handle labels, standard identifiers, integer numbers, the basic arithmetic operators (`+`, `-`, `*`, `/`), the assignment operator (`=`), the keywords `do`, `null`, and `repeat`, and the semicolon separator. Whitespace (spaces, tabs, newlines) must be ignored. No other token types are required yet. The public test case below shows the expected behaviour.

3.1 Public Test I.1

Input:

```
pp1: do x = 10 + y;
pp2: repeat z = x - 5;
```

Expected:

```
LABEL: pp1:
KEYWORD: do
IDENTIFIER: x
OPERATOR: =
NUMBER: 10
OPERATOR: +
IDENTIFIER: y
SEPARATOR: ;
LABEL: pp2:
KEYWORD: repeat
IDENTIFIER: z
OPERATOR: =
IDENTIFIER: x
OPERATOR: -
NUMBER: 5
SEPARATOR: ;
```

4 Part II: Structure and Types (3 marks)

Extend your lexer to recognise three additional constructs: data types, string literals, scope delimiters, and single-line comments. Specifically, add the data type keywords `integer`, `character`, and `string` (each as `DATATYPE`). Add string literals enclosed in double quotes, supporting the escape sequences \n, \t, \\, and \". The entire quoted string (including the quotes) becomes the lexeme for a `STRING` token. Add the square brackets `[` and `]` as `SCOPE_OPEN` and `SCOPE_CLOSE`. Finally, ignore single-line comments that begin with `//` and continue to the end of the line. The public test case illustrates these additions.

4.1 Public Test II.1

Input:

```
// Variable declaration
pp1: integer count = 0;
pp2: string msg = "hello\n";
pp3: do [ x = x + 1; ]
```

Expected:

```
LABEL: pp1:
DATATYPE: integer
IDENTIFIER: count
OPERATOR: =
NUMBER: 0
SEPARATOR: ;
LABEL: pp2:
DATATYPE: string
IDENTIFIER: msg
OPERATOR: =
STRING: "hello\n"
SEPARATOR: ;
LABEL: pp3:
KEYWORD: do
SCOPE_OPEN: [
IDENTIFIER: x
OPERATOR: =
IDENTIFIER: x
OPERATOR: +
NUMBER: 1
SEPARATOR: ;
SCOPE_CLOSE: ]
```

5 Part III: Advanced Operators (3 marks)

Now add the remaining operators. Recognise the multi-word keywords `in case that`, `otherwise`, and `jump to` as single `KEYWORD` tokens. Add the relational operators `gt`, `lt`, `gteq`, `lteq` as `RELOP`. Add the logical operators `and` and `or` as `LOGICAL`. Also add the modulo operator `%%` as an `OPERATOR`. The modulo operator consists of two percent signs with no space between them;

it is distinct from the single percent sign (which is not used in this language). The public test case shows how these new tokens appear.

5.1 Public Test III.1

Input:

```
pp1: in case that x gteq 10 and y lt 5 [
  result = x %% y;
]
pp2: otherwise jump to pp1:;
```

Expected:

```
LABEL: pp1:
KEYWORD: in case that
IDENTIFIER: x
RELOP: gteq
NUMBER: 10
LOGICAL: and
IDENTIFIER: y
RELOP: lt
NUMBER: 5
SCOPE_OPEN: [
IDENTIFIER: result
OPERATOR: =
IDENTIFIER: x
OPERATOR: %%
IDENTIFIER: y
SEPARATOR: ;
SCOPE_CLOSE: ]
LABEL: pp2:
KEYWORD: otherwise
KEYWORD: jump to
LABEL: pp1:
SEPARATOR: ;
```

6 Part IV: Ambiguity Handling (3 marks)

In this final part you must handle identifiers that can start with special characters and also the unary underscore operator. Special characters allowed at the beginning of an identifier are: `@`, `#`, `*`, `-`, `+`, `/`, `_`, `:`, and `\`. After the first character, only letters, digits, and underscores are permitted. The unary operator `_` (square root) is a separate token of type `UNARY_OP`. The lexer must use the longest match rule to decide between an identifier starting with underscore and a unary operator followed by another token. For example, `_x` is a single identifier because the pattern for an identifier that starts with underscore matches the entire string. Similarly, `x_2` is a single identifier (underscore appears in the middle), not `x` followed by unary operator on `2`. However, if whitespace separates them, e.g., `x _ 2`, then it becomes three tokens: identifier `x`, unary operator `_`, number `2`. Another ambiguity: the division operator `/` versus an identifier that starts with slash. The input `/abc` is an identifier, but `a / b` is identifier, operator, identifier. The longest match rule resolves this because `/abc` matches the identifier pattern (starting with

slash) while `a / b` has whitespace that breaks the identifier. Also ensure that keywords like `gt` are not part of longer identifiers: `gt1` must be an identifier, not the keyword `gt` followed by the number `1`. The public test case demonstrates the correct handling.

6.1 Public Test IV.1

Input:

```
pp1: /result = total / count;
pp2: _value = x_2 + _y;
pp3: @id1 = gt1;
```

Expected:

```
LABEL: pp1:
IDENTIFIER: /result
OPERATOR: =
IDENTIFIER: total
OPERATOR: /
IDENTIFIER: count
SEPARATOR: ;
LABEL: pp2:
IDENTIFIER: _value
OPERATOR: =
IDENTIFIER: x_2
OPERATOR: +
IDENTIFIER: _y
SEPARATOR: ;
LABEL: pp3:
IDENTIFIER: @id1
OPERATOR: =
IDENTIFIER: gt1
SEPARATOR: ;
```

7 Submission

Submit a ZIP file named `<roll_number>_lexer.zip` in capital letters containing:

- `lexer.l`
- `README.txt` (plain text, max 1 page)

Compilation:

```
flex lexer.l
gcc lex.yy.c -o lexer
```

(Add `-lfl` or `-ll` if needed, but better to define `yywrap()` in the file.)

8 Testing

Run on a test case:

```
./lexer < input.txt > output.txt
diff output.txt expected.txt
```

No output from `diff` means success.

9 Academic Integrity

No AI, no collaboration, no online resources except the `flex` manual. Violations will be penalised severely.

Good Luck

Implement incrementally. Start with Part I, test with the public case, then add Part II, and so on. A correct Part IV solution will pass all previous parts.